HIMANSHU

FULL STACK (MERN) DEVELOPER

+91 8826474169 | himanshurajuk04@gmail.com | github.com/himanshu-raj-uk

PROFESSIONAL SUMMARY

- Full Stack (MERN) Developer pursuing BCA from IGNOU, with hands-on experience building end-to-end, scalable web applications and REST APIs.
- Strong working knowledge of Node.js, Express.js, MongoDB, React.js and Tailwind CSS, with a focus on clean MVC architecture, mobile-first and desktop-ready UI, and secure authentication.
- Data protection focus: bcrypt.js password hashing, JWT role-based authorization and Mongoose schema validation. Built five projects, including three live websites: Service Planner, Skills Yard and PotterDB.

TECHNICAL SKILLS

Frontend: HTML5, CSS3, JavaScript (ES6+), TypeScript, React.js (Hooks, Context API), Tailwind CSS, Responsive & Mobile-First UI Design

Backend: Node.js, Express.js, MongoDB, Mongoose (schema design, validation & aggregation), REST API Development, JWT, bcrypt.js, Cloudinary, Multer

Auth & Architecture: JWT Authentication, Role-Based Authorization, Middleware Authentication, Password Hashing, MVC Architecture, CRUD Operations, Session & Token Management, Third-Party API Integration

Tools & Deployment: Git, GitHub, GitHub Pages, VS Code, Postman, Render, Vercel, Netlify, Vite, AI-Assisted Development

PROJECTS

Service Planner — *React.js, Tailwind CSS, Vite, Node.js, Express.js, MongoDB, Mongoose, JWT, bcrypt.js, Multer, Cloudinary*

Live Website: https://himanshu-raj-uk.github.io/Service-Planner/

Main project: live, publicly deployed full-stack planning web application ("Plan Everything Smartly") with a React frontend and a Node.js backend.

- **What it does:** Gives users one clean place to plan and organize their services, with secure accounts and persistent data, working fully in the browser with no installation.
- **How I built it:** Developed with AI coding assistants, then reviewed, refactored and tested every module line by line so the code stays clean, consistent and easy to maintain.
- **Backend architecture (MVC):** REST APIs built with Node.js and Express.js, organized into models, controllers, routes and reusable middleware, so each layer has one job and the code stays modular and easy to extend.
- **Registration and login flow:** Request is validated, existing users are checked, the password is hashed with bcrypt.js before saving, and on login the hash is compared with bcrypt and a signed JWT is issued to the client.
- **Protected API flow:** Route, then auth middleware (verifies the JWT and rejects invalid or missing tokens), then controller, then Mongoose model, then MongoDB, then a JSON response.
- **Models and validation:** Mongoose schemas with required fields, types, unique fields and validators, so input is verified on the frontend and checked again at the schema level before it reaches the database.
- **Media upload flow:** Multer receives the uploaded file on the server, the file is sent to Cloudinary, and the returned secure URL is saved in the model for fast, optimized delivery.
- **Frontend connection:** Component-based React pages call the backend REST APIs; the JWT from login is sent with protected requests, and private routes redirect unauthenticated users to Login. Endpoints were tested in Postman before connecting the UI.
- **Mobile and desktop UI:** Mobile-first Tailwind CSS design that adapts smoothly from phones to tablets and desktops, with readable fonts, touch-friendly controls and consistent spacing.
- **Deployment:** Created an optimized Vite production build with the correct base path and deployed the frontend live on GitHub Pages.

Netflix Clone — *React.js, TypeScript, Tailwind CSS, Node.js, Express.js, MongoDB, Mongoose*
GitHub: https://github.com/himanshu-raj-uk
Full-stack streaming platform where users register, log in and browse content, and admins manage the catalog from a dashboard.

- **What it does:** Complete Login and Registration flows, a protected browsing experience and an Admin Dashboard to add and manage content, with images and media delivered through Cloudinary.
- **Backend architecture (MVC):** REST APIs built with Node.js and Express.js, organized into models, controllers and routes with reusable middleware, so each layer has one job and the code stays modular and easy to extend.
- **Registration and login flow:** Request is validated, existing users are checked, the password is hashed with bcrypt.js before saving, and on login the hash is compared with bcrypt and a signed JWT is issued to the client.
- **Protected API flow:** Route, then auth middleware (verifies the JWT and rejects invalid or missing tokens), then controller, then Mongoose model, then MongoDB, then a JSON response. Admin routes get an extra role check.
- **Models and validation:** Mongoose schemas for users, content and sessions with required fields, types, unique fields and validators, so input is verified on the frontend and checked again at the schema level before it reaches the database.
- **Media upload flow:** Multer receives the uploaded file on the server, the file is sent to Cloudinary, and the returned secure URL is saved in the content model for fast, optimized delivery.
- **Frontend connection:** React and TypeScript pages call the backend REST APIs; the JWT from login is kept on the client and sent with protected requests, and private routes redirect unauthenticated users to Login. Endpoints were tested in Postman before connecting the UI.
- **Mobile and desktop UI:** Tailwind CSS mobile-first layout that reproduces a real streaming experience on phone, tablet and desktop.

Skills Yard — *HTML5, CSS3, Responsive Design*
Live Website: https://himanshu-raj-uk.github.io/Skills-Yard/
Multi-section, fully responsive landing website for an IT training institute, built with only HTML5 and CSS3.

- **What it does:** Presents the institute's courses, placement results and team to prospective students, with a clear Let's Begin call-to-action and a full navigation bar (Home, About, Course, Showcase, Events, Blogs, Contact).
- **Key sections built:** Hero banner; Why Choose Us cards (cutting-edge technologies, skilled instructors, hands-on learning, curriculum, internships and placements, one-to-one sessions); 10 course categories such as Web, App, Game Development, AI and ML, Data Science, Digital Marketing and VFX, each with its topic list.
- **Trust and engagement:** Placement statistics counters, student success-story cards with role and package, a How It Works trial flow, team member cards with Read More and Back toggle, testimonials, Our Latest Work banner, and a footer with quick links, top courses, address, email, phone and social icons.
- **How I built it:** Semantic HTML5 structure with Flexbox and CSS Grid layouts, custom CSS animations and hover effects, and no frameworks or libraries.
- **Mobile and desktop UI:** Mobile-first breakpoints so every section stacks cleanly on phones and expands into multi-column layouts on tablets and desktops.

PotterDB Data Explorer — *HTML5, CSS3, JavaScript (ES6+), Fetch API, REST API*
Live Website: https://himanshu-raj-uk.github.io/Potter-DB/
Multi-page data explorer for the Harry Potter universe that loads live data from the public PotterDB API.

- **What it does:** Lets users browse and search 7 books, 5,246 characters, 11 movies, 168 potions and 333 spells through dedicated pages (Books, Characters, Movies, Potions, Spells) plus a home page that explains the database and API.
- **How I built it:** Used the Fetch API with async/await to call multiple PotterDB REST endpoints and render the JSON response dynamically into interactive cards, with no page reloads.
- **Performance and usability:** Added Load More pagination so thousands of records load in small batches, search and filter options, a slide-in sidebar menu for quick navigation between categories, and loading and error states with clear UI feedback.

- **Mobile and desktop UI:** Responsive CSS Grid and Flexbox layout with a mobile-first design, so cards, header and sidebar work smoothly on every screen size.

E-Commerce Website — *React.js, Node.js, Express.js, MongoDB, Mongoose*

Complete MERN e-commerce application covering the customer storefront and full admin operations.

- **What it does:** Customers browse products and categories, use a Shopping Cart and Wishlist that persist per user, and place orders; staff manage everything from an Admin Dashboard.
- **Backend architecture (MVC):** Node.js and Express.js APIs split into models, controllers, routes and middleware, with Mongoose models for Users, Products, Categories and Orders.
- **Authentication flow:** Passwords are hashed with bcrypt.js on registration and compared on login; a JWT is issued and verified by custom middleware on every protected route.
- **Role-based authorization:** Separate User, Admin and Employee roles; middleware checks the role from the token before admin or employee actions such as managing products, orders and staff.
- **Double verification of data:** Frontend form checks first, then Mongoose schema validation and validated REST APIs on the backend, so bad or unsafe input never reaches the database.
- **Image upload flow:** Multer handles product image uploads, Cloudinary stores them, and the returned URL is saved with the product document.
- **Frontend connection:** React pages call the REST APIs for products, cart, wishlist, orders and admin actions, sending the JWT with protected requests; cart and wishlist state is saved per user in the database.
- **Admin Dashboard and UI:** Manage products, categories, orders and employees from a responsive, mobile-first interface that scales up for desktop.

EDUCATION

BCA (Pursuing) — IGNOU, Delhi

Class XII — CBSE Board

Class X — CBSE Board

HOBBIES

Building Web Applications • Learning New Technologies • Reading Tech Articles • Listening to Music